

Phase_D_Technology_Architecture

TOGAF ADM - Phase D: Technology Architecture

Purpose: Define the technology infrastructure needed to support applications and data

Overview

PHASE D: TECHNOLOGY ARCHITECTURE

"What hardware, software, and infrastructure do we need?"

Key Activities

1. **Define technology platforms** - What infrastructure will support our applications
2. **Map applications to technology** - Which app runs on which platform
3. **Perform Gap Analysis** - What technology changes are needed

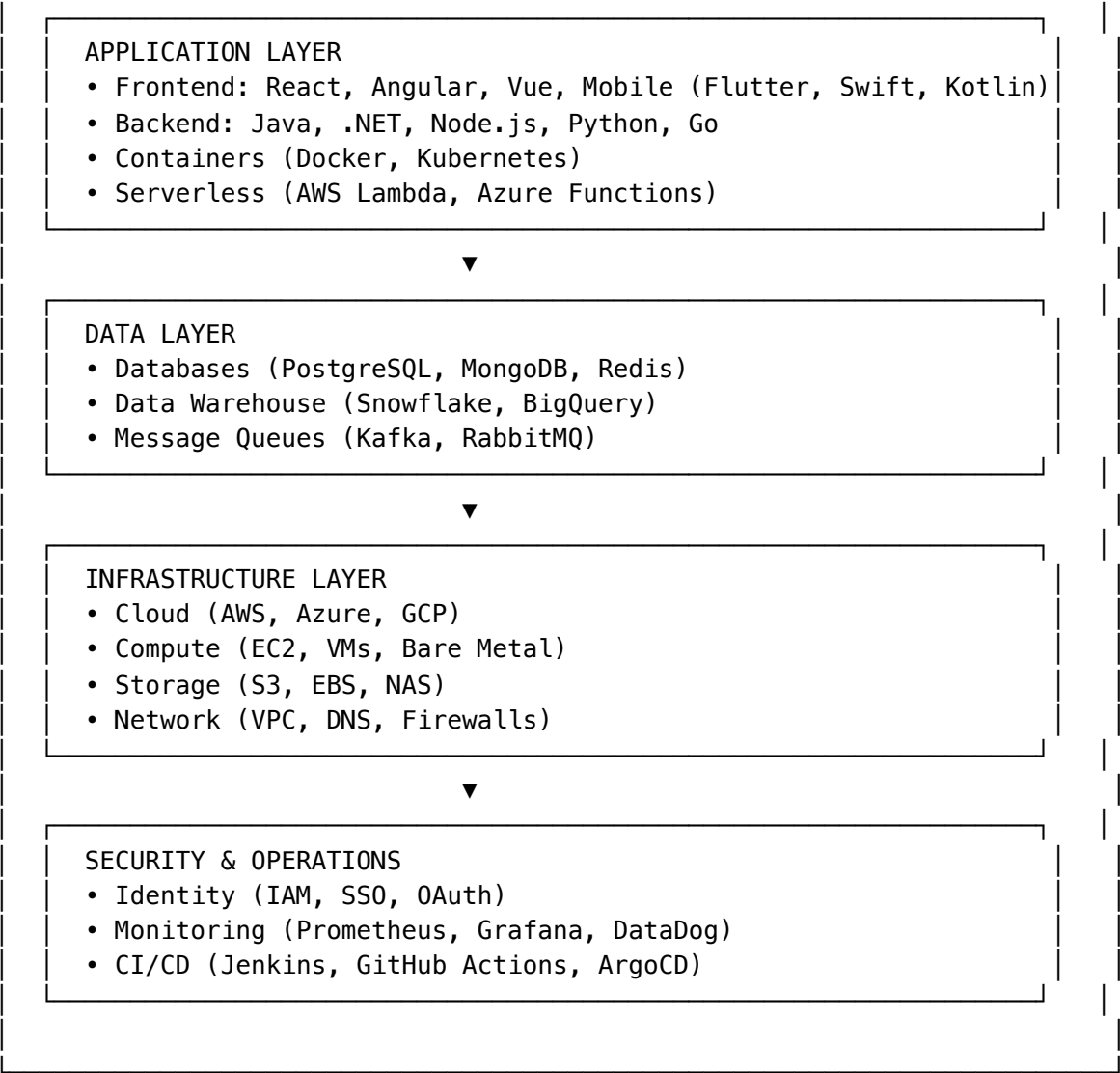
Technology Platform Layers

TECHNOLOGY PLATFORM LAYERS

EDGE / DELIVERY LAYER (Entry Point for Users)

- CDN (CloudFront, Akamai) - Cache & deliver static content
- Load Balancers (ALB, HAProxy) - Distribute traffic
- Web servers (Nginx, Apache) - Serve requests
- API Gateway - Route API calls





Application to Technology Mapping

APPLICATION ↔ TECHNOLOGY MAPPING MATRIX					
APPLICATION	COMPUTE	DATABASE	RUNTIME	CLOUD	CONTAINER
Player Portal	EKS	PostgreSQL	Node.js	AWS	✓
Payment Service	EKS	PostgreSQL	Java	AWS	✓
Game Engine	EC2	Redis	C++	AWS	x
Analytics	EMR	Redshift	Python	AWS	x
Admin Portal	Lambda	DynamoDB	Node.js	AWS	x
Legacy ERP	On-Prem	Oracle	Java	N/A	x

Layer Breakdown

Edge / Delivery Layer

EDGE / DELIVERY LAYER	
PURPOSE: Entry point for all user requests	
COMPONENTS:	
CDN	Cache static content close to users Examples: CloudFront, Akamai, Cloudflare
Load Balancer	Distribute traffic across multiple servers Examples: ALB, NLB, HAProxy, Nginx
Web Server	Serve HTTP requests, SSL termination Examples: Nginx, Apache
API Gateway	Route, authenticate, rate-limit API calls Examples: Kong, AWS API Gateway, Apigee

Application Layer

APPLICATION LAYER	
PURPOSE: Where your applications run	
FRONTEND:	
• Web: React, Angular, Vue.js • Mobile: Flutter, Swift, Kotlin, React Native	
BACKEND:	
• Languages: Java, .NET, Node.js, Python, Go • Frameworks: Spring Boot, Express, Django, FastAPI	
DEPLOYMENT:	
• Containers: Docker + Kubernetes (EKS, AKS, GKE) • Serverless: AWS Lambda, Azure Functions, Cloud Functions • VMs: EC2, Azure VMs, Compute Engine	

Data Layer

DATA LAYER**DATABASES:**

- Relational: PostgreSQL, MySQL, Oracle, SQL Server
- NoSQL: MongoDB, DynamoDB, Cassandra
- In-Memory: Redis, Memcached
- Time-Series: InfluxDB, TimescaleDB

DATA WAREHOUSE:

- Snowflake, BigQuery, Redshift, Synapse

MESSAGING:

- Event Streaming: Kafka, Kinesis, Pulsar
- Message Queue: RabbitMQ, SQS, Azure Service Bus

Infrastructure Layer

INFRASTRUCTURE LAYER**CLOUD PROVIDERS:**

- AWS, Azure, GCP, Oracle Cloud

COMPUTE:

- Virtual Machines (EC2, Azure VMs)
- Container Orchestration (EKS, AKS, GKE)
- Bare Metal (for high-performance needs)

STORAGE:

- Object: S3, Azure Blob, GCS
- Block: EBS, Azure Disk
- File: EFS, Azure Files

NETWORKING:

- VPC, Subnets, Security Groups
- DNS (Route53, Azure DNS)
- Firewalls, WAF

Security & Operations

SECURITY & OPERATIONS

- IDENTITY & ACCESS:

 - IAM, SSO, OAuth 2.0, SAML
 - Secrets Management: HashiCorp Vault, AWS Secrets Manager

MONITORING & OBSERVABILITY:

 - Metrics: Prometheus, CloudWatch, DataDog
 - Logging: ELK Stack, Splunk, CloudWatch Logs
 - Tracing: Jaeger, X-Ray, Zipkin
 - Dashboards: Grafana, Kibana

CI/CD:

 - Build: Jenkins, GitHub Actions, GitLab CI
 - Deploy: ArgoCD, Spinnaker, Flux
 - Artifact: Nexus, Artifactory, ECR

Key Deliverables

Deliverable	Description
Technology Architecture Document	Complete technology architecture
Platform Decomposition Diagram	Layers of technology stack
Application/Technology Matrix	Which app runs on what
Technology Standards Catalog	Approved technologies
Gap Analysis	What technology changes needed

Summary

- PHASE D = "WHAT INFRASTRUCTURE DO WE NEED?"

Define technology across layers:

 - Edge / Delivery (CDN, Load Balancer, API Gateway)
 - Application (Frontend, Backend, Containers)
 - Data (Databases, Warehouses, Messaging)
 - Infrastructure (Cloud, Compute, Storage, Network)
 - Security & Ops (IAM, Monitoring, CI/CD)

OUTPUT: Technology Architecture Document + Gap Analysis